

Personal Expense Tracker — Submission Package

Writeup

- **Project name:** Personal Expense Tracker (CLI)
- **Description:** Simple command-line application to set a monthly budget (INR), add expenses, view a categorized spending summary, and reset stored data. Data is persisted to `expenses.json` in the same folder.
- **Languages / Libraries:** Python 3.x, standard library only (`json`, `os`, `re`, `datetime`).
- **Features:**
 - Persistent storage in UTF-8 JSON
 - Budget in Indian Rupees (₹)
 - Category validation: categories must contain at least one alphabetic character (prevents purely numeric keys)
 - Reset option (overwrite file or remove file)
 - Summary grouped by category

Run / Submission Instructions

1. Save `ExpenseTracker.py` to the same folder as this file (already included below).
2. Open a terminal in the folder (Windows example):

```
```powershell
cd C:\AIML\D1
py ExpenseTracker.py
```
```

3. Use the menu (1-5) to set budget, add expenses, view summary, reset data, or exit.

Notes:

- The program creates `expenses.json` in the same folder. Keep the file smaller than 10 MB for submission.
- Use option 5 to reset data (you will be prompted to confirm).

Screenshots (attach these files when submitting)

- `expense\_summary.png` — screenshot of the CLI after adding sample expenses and selecting "View Spending Summary".
- `add\_expense\_input.png` — screenshot of adding an expense sequence.
- `expenses\_json.png` — screenshot of the `expenses.json` file open in an editor.

Each screenshot should be PNG/JPG and less than 10 MB.

Source Code (single file)

Save the following as `ExpenseTracker.py` (it's included here for convenience):

```
```python
#!/usr/bin/env python3
import json
import os
import re
from datetime import datetime

class ExpenseTracker:
 def __init__(self, filename="expenses.json"):
 self.filename = filename
 self.expenses = []
 self.budget = 0.0
 self.load_data()

 def is_valid_category(self, category):
 """Category must contain at least one alphabetic character."""
 if not isinstance(category, str):
 return False
 if not re.search("[a-zA-Z]", category):
 return False
 return True
```

```

 return False
 category = category.strip()
 if not category:
 return False
 return bool(re.search(r"[A-Za-z]", category))

--- FILE HANDLING ---
def load_data(self):
 """Loads data from the JSON file into memory."""
 try:
 with open(self.filename, "r", encoding="utf-8") as file:
 data = json.load(file)
 self.budget = float(data.get("budget", 0.0))
 self.expenses = data.get("expenses", [])
 except FileNotFoundError:
 self.budget = 0.0
 self.expenses = []
 except (json.JSONDecodeError, ValueError):
 print("Warning: Save file is corrupted or invalid. Starting fresh.")
 self.budget = 0.0
 self.expenses = []

def save_data(self):
 """Saves the current state to the JSON file."""
 data_to_save = {
 "budget": self.budget,
 "expenses": self.expenses
 }
 with open(self.filename, "w", encoding="utf-8") as file:
 json.dump(data_to_save, file, indent=4, ensure_ascii=False)

def reset_data(self, remove_file=False):
 """Reset in-memory data and overwrite save file. If remove_file=True, remove the file."""
 self.budget = 0.0
 self.expenses = []
 if remove_file:
 try:
 os.remove(self.filename)
 print("\n■ All data file removed.")
 except FileNotFoundError:
 pass
 except OSError:
 print("\n■■ Could not remove the data file; overwriting instead.")
 self.save_data()
 else:
 self.save_data()
 print("\n■ All data has been reset.")

def set_budget(self, amount):
 self.budget = float(amount)
 self.save_data()
 print(f"\n■ Monthly budget successfully set to: ■{self.budget:.2f}")

def add_expense(self, category, amount, description):
 # normalize and validate category
 category = str(category).strip()
 if not self.is_valid_category(category):
 raise ValueError("Category must contain at least one letter (no purely numeric categories).")

```

```

category = category.title()
expense = {
 "date": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
 "category": category,
 "amount": float(amount),
 "description": description
}
self.expenses.append(expense)
self.save_data()
print("\n■ Expense added successfully!")

```

```

def view_summary(self):
 if not self.expenses and self.budget == 0:
 print("\n■■ No data found. Please set a budget and add expenses first.")
 return

```

```

total_spent = sum(float(item.get("amount", 0)) for item in self.expenses)
remaining = self.budget - total_spent

```

```

print("\n" + "="*30)
print("■ EXPENSE SUMMARY")
print("="*30)
print(f"Total Budget: ■{self.budget:.2f}")
print(f"Total Spent: ■{total_spent:.2f}")

```

```

if remaining < 0:
 print(f"Remaining: -■{abs(remaining):.2f} (OVER BUDGET!)")
else:
 print(f"Remaining: ■{remaining:.2f}")

```

```

if self.expenses:
 print("\n■ SPENDING BY CATEGORY:")
 category_totals = {}
 for exp in self.expenses:
 cat = exp.get("category", "Unknown")
 amt = float(exp.get("amount", 0.0))
 category_totals[cat] = category_totals.get(cat, 0.0) + amt

 for cat, amount in category_totals.items():
 print(f" - {cat}: ■{amount:.2f}")
 print("="*30)

```

```

def main():
 tracker = ExpenseTracker()

```

```

while True:
 print("\n" + "-"*30)
 print(" PERSONAL EXPENSE TRACKER")
 print("-"*30)
 print("1. Set Monthly Budget")
 print("2. Add a New Expense")
 print("3. View Spending Summary")
 print("4. Exit")
 print("5. Reset All Data")

```

```

choice = input("\nChoose an option (1-5): ").strip()

```

```

if choice == '1':
 try:

```

```

 amount = float(input("Enter your monthly budget: █"))
 if amount < 0:
 print("█ Budget cannot be negative.")
 else:
 tracker.set_budget(amount)
 except ValueError:
 print("█ Invalid input. Please enter a number.")

elif choice == '2':
 category = input("Enter category (e.g., Food, Rent, Utilities): ").strip()
 if not tracker.is_valid_category(category):
 print("█ Invalid category. Must contain at least one letter (no purely numeric
categories).")
 continue

 try:
 amount = float(input("Enter amount: █"))
 if amount <= 0:
 print("█ Amount must be greater than zero.")
 continue
 except ValueError:
 print("█ Invalid input. Please enter a number.")
 continue

 desc = input("Enter a brief description: ").strip()
 try:
 tracker.add_expense(category, amount, desc)
 except ValueError as e:
 print(f"█ {e}")

elif choice == '3':
 tracker.view_summary()

elif choice == '4':
 print("\nSaving data... Goodbye!")
 break

elif choice == '5':
 confirm = input("Type 'RESET' to permanently delete all data, or 'RESET FILE' to
remove the file, else press Enter to cancel: ").strip()
 if confirm == 'RESET FILE':
 tracker.reset_data(remove_file=True)
 elif confirm == 'RESET':
 tracker.reset_data(remove_file=False)
 else:
 print("Canceled.")

else:
 print("█ Invalid choice. Please select 1, 2, 3, 4, or 5.")

if __name__ == "__main__":
 main()
...

Sample `expenses.json` (optional to attach)
```json
{
    "budget": 50000.0,
    "expenses": [
        {"date": "2026-06-11 10:00:00", "category": "Rent", "amount": 10000, "description":

```

```
"June rent"},
  {"date": "2026-06-11 11:00:00", "category": "Food", "amount": 10000, "description":
"Groceries"}
]
}
```

Links

- If hosted on GitHub, paste the repo URL here (example):
 - ``https://github.com/<your-username>/personal-expense-tracker``

Additional Remarks (<=1200 chars)

This lightweight CLI tracks expenses and budget in INR. No third-party dependencies required. Data is stored locally in ``expenses.json`` (UTF-8). Category validation prevents accidental numeric-only categories. Reset option supports overwriting or removing the file. Tested on Windows using Python launcher; works in VS Code integrated terminal. For submission, attach this markdown file plus the screenshots and the source file (or a ZIP). I can also produce unit tests or a short demo video on request.